

7. Consultas resumen.

7.1. Introducción.....	1
7.2. Funciones de agregación.....	1
7.2.1. Contar valores distintos COUNT(DISTINCT columna).....	2
7.3. Agrupamiento de filas (GROUP BY).....	2
7.4. Condición de agrupamiento (HAVING).....	3
7.5. Algunos errores:.....	3
7.5.1. Error al contar el número de filas distintas.....	3
7.5.2. Error al intentar utilizar una función de agregación en la cláusula WHERE.....	4
7.5.3. Error al usar COUNT(*) y COUNT(columna) al hacer un LEFT JOIN.....	5
7.6. Ejercicios.....	6
7.6.1. Base de datos: tiendadb.....	6
7.6.2. Base de datos: empleadodb.....	7
7.6.3. Base de datos: ventasdb.....	8

7.1. Introducción.

Recordando la sintaxis para realizar una consulta con la sentencia **SELECT** de MySQL:

```
SELECT [DISTINCT] expresión_select [, expresión_select ...]  
    [FROM tabla]  
    [WHERE condición_where]  
    [GROUP BY {nombre_columna | expresión | posición} [ASC | DESC], ... [WITH  
        ROLLUP]]  
    [HAVING condición_where]  
    [ORDER BY {nombre_columna | expresión | posición} [ASC | DESC], ...]  
    [LIMIT {[offset,] row COUNT | row COUNT OFFSET offset}]
```

Es muy importante conocer **en qué orden se ejecuta cada una de las cláusulas** que forman la sentencia **SELECT**. El orden de ejecución es el siguiente:

1. Cláusula **FROM**.
2. Cláusula **WHERE** (Es opcional, puede ser que no aparezca).
3. Cláusula **GROUP BY** (Es opcional, puede ser que no aparezca).
4. Cláusula **HAVING** (Es opcional, puede ser que no aparezca).
5. Cláusula **SELECT**.
6. Cláusula **ORDER BY** (Es opcional, puede ser que no aparezca).
7. Cláusula **LIMIT** (Es opcional, puede ser que no aparezca).

En esta unidad vamos a trabajar con dos nuevas cláusulas **GROUP BY** y **HAVING**.

7.2. Funciones de agregación.

Estas funciones realizan una operación específica sobre todas las filas de un grupo, definido por **GRUPO BY** o en su defecto afectando a todas las filas a mostrar. Las funciones de agregación más comunes son:

MAX (expr)	Valor máximo del grupo
MIN (expr)	Valor mínimo del grupo
SUM (expr)	Suma de todos los valores del grupo

AVG(expr)	Valor medio del grupo
COUNT(*)	Número de filas que tiene el resultado de la consulta incluido NULL
COUNT(columna)	Número de valores no nulos que hay en esa columna

Importante: Las funciones de agregación sólo se pueden usar en las cláusulas **SELECT** y **GROUP BY**.

Las funciones **COUNT(*)** y **COUNT(columna)** devolverán resultados diferentes cuando haya valores nulos en la columna que estamos usando en la función.

Ejemplos:

Supongamos que tenemos los siguientes valores en la tabla **alumno**:

id	nombre	apellido1	apellido2	fecha_nacimiento	es_repetidor	teléfono
1	María	S-anchez	Pérez	1990/12/01	no	NULL
2	Juan	Sáez	Vega	1998/05/02	no	618253876
3	Pepe	Ramírez	Gea	1988/01/03	no	NULL
4	Lucía	López	Ruiz	1993/06/13	sí	678516294

La consulta:

```
mysql>SELECT COUNT(teléfono) FROM alumno;
```

Devolverá el valor 2, solo hay dos teléfonos no nulos. Mientras que la consulta:

```
mysql>SELECT COUNT(*) FROM alumno;
```

Devolverá el valor 4, porque hay cuatro filas que mostrar.

7.2.1. Contar valores distintos COUNT(DISTINCT columna)

Supongamos que tenemos los siguientes valores en la tabla **producto**:

id	nombre	precio	código_fabricante
1	Disco duro SATA3 1TB	86	5
2	Memoria RAM DDR4 8GB	120	4
3	Disco SSD 1 TB	150	5
4	GeForce GTX 1050Ti	185	5

Y nos piden calcular el número de valores distintos de código de fabricante que aparecen en la tabla **producto**.

```
mysql>SELECT COUNT(DISTINCT código_fabricante) FROM producto;
```

Esta consulta devolverá: 2

7.3. Agrupamiento de filas (GROUP BY)

La cláusula **GROUP BY** nos permite crear **grupos de filas** que tienen los mismos valores en las columnas por las que se desea agrupar.

En la siguiente captura se han agrupado los productos de una tienda por código_fabricante y se muestra el producto más barato de cada fabricante: min(precio).

```
mysql> select * from producto order by codigo_fabricante;
```

codigo	nombre	precio	codigo_fabricante
6	Monitor 24 LED Full HD	207	1
7	Monitor 27 LED Full HD	250.99	1
8	Portátil Yoga 520	564	2
9	Portátil Ideapad 320	449	2
10	Impresora HP Deskjet 3720	64.99	3
11	Impresora HP Laserjet Pro M26nw	185	3
3	Disco SSD 1 TB	155.99	4
12	Portátil i7	680	4
1	Disco duro SATA3 1TB	91.99	5
2	Memoria RAM DDR4 8GB	125	6
5	GeForce GTX 1080 Xtreme	760	6
4	GeForce GTX 1050Ti	190	7
13	Disco SSD 480 GB	87	8
14	Disco SSD 1TB	175	16
15	Disco SSD 256 GB	50	16

15 rows in set (0.00 sec)


```
mysql> select codigo_fabricante as fabr, min(precio)
from producto group by codigo_fabricante;
```

fabr	min(precio)
1	207
2	449
3	64.99
4	155.99
5	91.99
6	125
7	190
8	87
16	50

9 rows in set (0.00 sec)

7.4. Condición de agrupamiento (HAVING)

La cláusula **HAVING** nos permite crear **filtros** sobre los grupos de filas que tienen los mismos valores en las columnas por las que se ha agrupado.

En la siguiente captura se han filtrado los fabricantes que tienen un precio mínimo inferior a 100. Para ello se ha de agrupar primero por código_fabricante y mostrar solo los que tengan un precio mínimo superior a 100.

```
mysql> select * from producto order by codigo_fabricante;
```

codigo	nombre	precio	codigo_fabricante
6	Monitor 24 LED Full HD	207	1
7	Monitor 27 LED Full HD	250.99	1
8	Portátil Yoga 520	564	2
9	Portátil Ideapad 320	449	2
10	Impresora HP Deskjet 3720	64.99	3
11	Impresora HP Laserjet Pro M26nw	185	3
3	Disco SSD 1 TB	155.99	4
12	Portátil i7	680	4
1	Disco duro SATA3 1TB	91.99	5
2	Memoria RAM DDR4 8GB	125	6
5	GeForce GTX 1080 Xtreme	760	6
4	GeForce GTX 1050Ti	190	7
13	Disco SSD 480 GB	87	8
14	Disco SSD 1TB	175	16
15	Disco SSD 256 GB	50	16

15 rows in set (0.00 sec)


```
mysql> select codigo_fabricante as fabr, min(precio)
from producto group by codigo_fabricante
having min(precio)>100;
```

fabr	min(precio)
1	207
2	449
4	155.99
6	125
7	190

5 rows in set (0.00 sec)

7.5. Algunos errores:

7.5.1. Error al contar el número de filas distintas

Supongamos que queremos saber el número de proveedores que tenemos. Es decir, el número de códigos de fabricantes distintos que tenemos en la tabla de productos.

```
mysql> SELECT DISTINCT COUNT(codigo_fabricante) FROM producto;
```

Nos devolvería en total de productos que tenemos, pues contaría el número de filas que tienen código_fabricante. Como todos los productos tienen un fabricante, luego el número total de productos.

```
mysql> SELECT COUNT( DISTINCT codigo_fabricante) FROM producto;
```

En este caso solo mostraría los códigos de fabricantes que son distintos. Luego si nos ofrece el total de fabricantes que son proveedores de la tienda.

```
mysql> select * from producto order by codigo_fabricante;
```

codigo	nombre	precio	codigo_fabricante
6	Monitor 24 LED Full HD	207	1
7	Monitor 27 LED Full HD	250.99	1
8	Portátil Yoga 520	564	2
9	Portátil Ideapd 320	449	2
10	Impresora HP Deskjet 3720	64.99	3
11	Impresora HP Laserjet Pro M26nw	185	3
3	Disco SSD 1 TB	155.99	4
12	Portátil i7	680	4
1	Disco duro SATA3 1TB	91.99	5
2	Memoria RAM DDR4 8GB	125	6
5	GeForce GTX 1080 Xtreme	760	6
4	GeForce GTX 1050Ti	190	7
13	Disco SSD 480 GB	87	8
14	Disco SSD 1TB	175	16
15	Disco SSD 256 GB	50	16

```
15 rows in set (0.00 sec)
```

```
mysql> SELECT DISTINCT COUNT(codigo_fabricante) FROM producto;
```

COUNT(codigo_fabricante)
15

```
1 row in set (0.00 sec)
```

```
mysql> SELECT COUNT(DISTINCT codigo_fabricante) FROM producto;
```

COUNT(DISTINCT codigo_fabricante)
9

```
1 row in set (0.00 sec)
```

7.5.2. Error al intentar utilizar una función de agregación en la cláusula WHERE.

Las funciones de agregación o resumen sólo se pueden usar en las cláusulas **SELECT** Y **HAVING**, por lo tanto si intenta hacer uso de una función de agregación en la cláusula **WHERE** obtendrá un error.

Ejemplo: Devuelve un listado con los productos que tienen un precio superior al precio medio de todos los artículos que existen en la tabla **productos**.

```
mysql> select nombre, precio from producto where precio>=avg(precio);
ERROR 1111 (HY000): Invalid use of group function
```

Uso inapropiado de función resumen. AVG() solo se puede usar con:

- SELECT
- HAVING

```
mysql> select nombre, precio from producto
where precio>=(select avg(precio) from producto);
```

nombre	precio
GeForce GTX 1080 Xtreme	760
Portátil Yoga 520	564
Portátil Ideapd 320	449
Portátil i7	680

```
4 rows in set (0.00 sec)
```

La única forma de resolver este ejercicio es con una subconsulta. Cuando en una consulta se hace otra consulta. La segunda consulta está entre paréntesis.

7.5.3. Error al usar COUNT(*) y COUNT(columna) al hacer un LEFT JOIN

Realicemos una consulta que devuelva un listado con el número de productos que tiene cada fabricante. El listado debe incluir aquellos fabricantes que no tienen productos asociados indicando que tienen 0 productos.

En este caso es necesario realizar un **RIGHT JOIN** con las tablas **producto** y **fabricante**, de esa forma incluirá los fabricantes que no están relacionados con los artículos.

Por ejemplo, al ejecutar la consulta utilizando **count(*)** nos aparece que los fabricantes **Huawei**, **Acer**, **IBM** y **Xiaomi** tienen un producto asociado cada uno, cosa que no es así, pues estos proveedores no tienen ningún producto asociado.

Pero **count(*)** ha contado las filas en las que **p.codigo_fabricante** es **NULL**.

```
mysql> select f.nombre, count(*)
       from producto as p right join fabricante as f on p.codigo_fabricante=f.codigo
       group by f.codigo;
```

nombre	count(*)
Asus	2
Lenovo	2
Hewlett-Packard	2
Samsung	2
Seagate	1
Crucial	2
Gigabyte	1
Apple	1
Xiaomi	1
IBM	1
Acer	1
Dell	2
Huawei	1

13 rows in set (0.00 sec)

Sin embargo, si utilizamos **count(p.codigo_fabricante)** en este caso no cuenta las filas **NULL** por lo que aparecen con 0 productos.

```
mysql> select f.nombre, count(p.codigo_fabricante)
       from producto as p right join fabricante as f on p.codigo_fabricante=f.codigo
       group by f.codigo;
```

nombre	count(p.codigo_fabricante)
Asus	2
Lenovo	2
Hewlett-Packard	2
Samsung	2
Seagate	1
Crucial	2
Gigabyte	1
Apple	1
Xiaomi	0
IBM	0
Acer	0
Dell	2
Huawei	0

13 rows in set (0.00 sec)

Para contar el número de productos que tiene cada fabricante es necesario realizar un **GROUP BY** por el código del fabricante y contar el número de filas que tiene cada uno de los grupos que hemos creado. Pero debemos tener cuidado porque obtendremos resultados diferentes al contar el número de filas de cada grupo con **COUNT (*)** y **COUNT(p.codigo_fabricante)**. La opción correcta es hacer uso de **COUNT(p.codigo_fabricante)** porque contará aquellas filas que tienen un valor distinto de **NULL** en la columna **p.codigo_fabricante**.

7.6. Ejercicios.

7.6.1. Base de datos: tiendadb.

Consultas sobre la base de datos: Tienda informática.

La descripción de las tablas utilizadas es:

```
mysql> DESC producto;
```

Field	Type	Null	Key	Default	Extra
codigo	int(10) unsigned	NO	PRI	NULL	auto_increment
nombre	varchar(100)	NO		NULL	
precio	double	NO		NULL	
codigo_fabricante	int(10) unsigned	NO	MUL	NULL	

4 rows in set (0.01 sec)

```
mysql> DESC fabricante;
```

Field	Type	Null	Key	Default	Extra
codigo	int(10) unsigned	NO	PRI	NULL	auto_increment
nombre	varchar(100)	NO		NULL	

2 rows in set (0.00 sec)

Partimos de los siguientes contenidos de las tablas:

```
mysql> SELECT * FROM producto;
```

codigo	nombre	precio	codigo_fabricante
1	Disco duro SATA3 1TB	86.99	5
2	Memoria RAM DDR4 8GB	120	6
3	Disco SSD 1 TB	150.99	4
4	GeForce GTX 1050Ti	185	7
5	GeForce GTX 1080 Xtreme	755	6
6	Monitor 24 LED Full HD	202	1
7	Monitor 27 LED Full HD	245.99	1
8	Portátil Yoga 520	559	2
9	Portátil Ideapad 320	444	2
10	Impresora HP Deskjet 3720	59.99	3
11	Impresora HP Laserjet Pro M26nw	180	3

11 rows in set (0.00 sec)

```
mysql> SELECT * FROM fabricante;
```

codigo	nombre
1	Asus
2	Lenovo
3	Hewlett-Packard
4	Samsung
5	Seagate
6	Crucial
7	Gigabyte
8	Huawei
9	Xiaomi

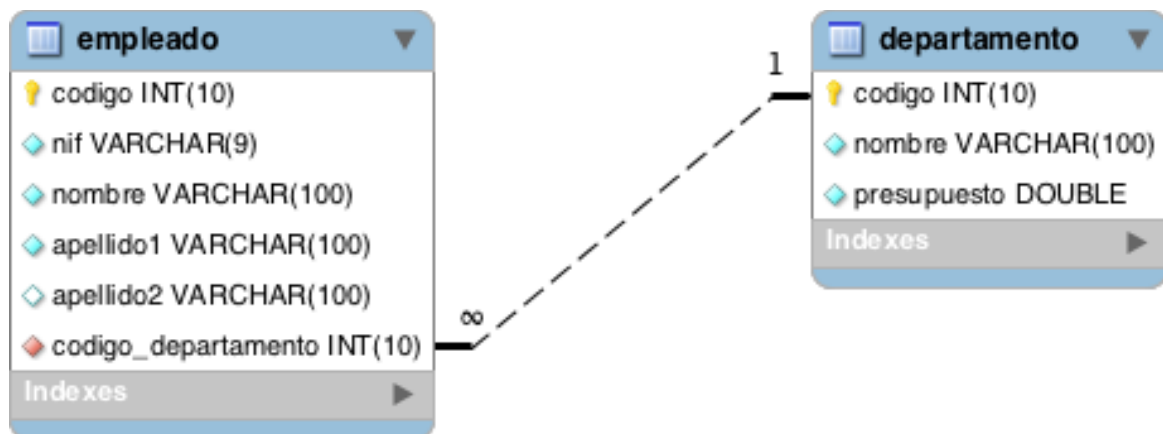
9 rows in set (0.00 sec)

1. Calcula el número total de fabricantes que hay en la tabla fabricante.
2. Calcula el número de valores distintos de código de fabricante aparecen en la tabla productos.
3. Calcula el precio medio, el máximo y el mínimo de todos los fabricantes.
4. Lista el nombre y el precio del producto más caro.
5. Calcula el número de productos que tiene el fabricante Asus.
6. Calcula la media del precio de todos los productos del fabricante Asus.
7. Calcula el precio más barato de todos los productos del fabricante Asus.
8. Muestra el precio máximo, precio mínimo, precio medio y el número total de productos que tiene el fabricante Crucial.

- 9 Muestra el número total de productos que tiene cada uno de los fabricantes. El listado también debe incluir los fabricantes que no tienen ningún producto. El resultado mostrará dos columnas, una con el nombre del fabricante y otra con el número de productos que tiene. Ordene el resultado descendientemente por el número de productos.
- 10 Muestra el nombre de cada fabricante, junto con el precio máximo, precio mínimo, precio medio y el número total de productos de los fabricantes que tienen un precio medio superior a 200€. Es necesario mostrar el nombre del fabricante.
- 11 Calcula el número de productos que tienen un precio mayor o igual a 180€.
- 12 Calcula el número de productos que tiene cada fabricante con un precio mayor o igual a 180€.
- 13 Lista el precio medio los productos de cada fabricante, mostrando solamente el código del fabricante.
- 14 Lista el precio medio los productos de cada fabricante, mostrando solamente el nombre del fabricante.
- 15 Lista los nombres de los fabricantes cuyos productos tienen un precio medio mayor o igual a 150€.
- 16 Devuelve un listado con los nombres de los fabricantes que tienen 2 o más productos.
- 17 Devuelve un listado con los nombres de los fabricantes y el número de productos que tiene cada uno con un precio superior o igual a 220 €. No es necesario mostrar el nombre de los fabricantes que no tienen productos que cumplan la condición.
- 18 Devuelve un listado con los nombres de los fabricantes y el número de productos que tiene cada uno con un precio superior o igual a 220 €. El listado debe mostrar el nombre de todos los fabricantes, es decir, si hay algún fabricante que no tiene productos con un precio superior o igual a 220€ deberá aparecer en el listado con un valor igual a 0 en el número de productos. (ha de hacerse con UNION).
- 19 Devuelve un listado con los nombres de los fabricantes donde la suma del precio de todos sus productos es superior a 1000 €.

7.6.2. Base de datos: empleadodb.

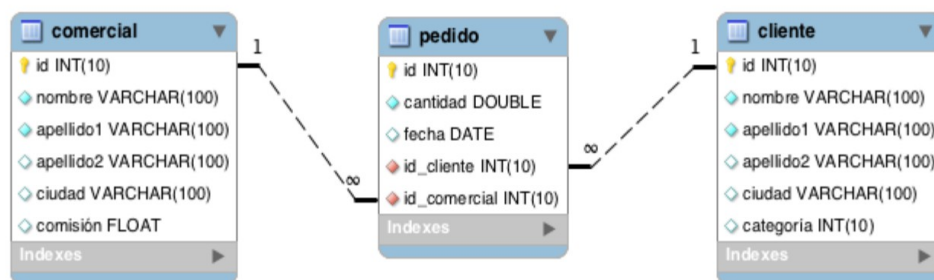
Modelo entidad/relación:



1. Calcula la suma, media, máximo y mínimo del presupuesto de todos los departamentos.
2. Calcula el nombre del departamento con menor presupuesto. Muestra el departamento y su presupuesto.
3. Calcula el valor máximo del presupuesto de todos los departamentos.
4. Calcula el nombre del departamento con mayor presupuesto. Muestra el departamento y su presupuesto.
5. Calcula el número total de empleados que hay en la tabla `empleado`.
6. Calcula el número de empleados que **no tienen** `NULL` en su segundo apellido.
7. Calcula el número de empleados que hay en cada departamento. Tienes que devolver dos columnas, una con el nombre del departamento y otra con el número de empleados que tiene asignados.
8. Calcula el nombre de los departamentos que tienen más de 2 empleados. El resultado debe tener dos columnas, una con el nombre del departamento y otra con el número de empleados que tiene asignados.
9. Calcula el número de empleados que trabajan en cada uno de los departamentos. El resultado de esta consulta también tiene que incluir aquellos departamentos que no tienen ningún empleado asociado.
10. Calcula el número de empleados que trabajan en cada uno de los departamentos que tienen un presupuesto mayor a 2000 euros.

7.6.3. Base de datos: ventasdb.

La base de datos `ventasdb` sigue el siguiente diagrama de Entidad-Relación:



1. Calcula la cantidad total que suman todos los pedidos, la media de los pedidos, el valor máximo de pedido y el valor del pedido mínimo que aparecen en la tabla pedido.
2. Calcula el número total de comerciales distintos que aparecen en la tabla pedido.
3. Calcula el número total de clientes que aparecen en la tabla cliente.
4. Calcula cuál es el valor máximo de categoría para cada una de las ciudades que aparece en la tabla cliente.

5. Calcula cuál es el máximo valor de los pedidos realizados durante el mismo día para cada uno de los clientes. Es decir, el mismo cliente puede haber realizado varios pedidos de diferentes cantidades el mismo día. Se pide que se calcule cuál es el pedido de máximo valor para cada uno de los días en los que un cliente ha realizado un pedido. Muestra el identificador del cliente, nombre, apellidos, la fecha y el valor de la cantidad.
6. Calcula cuál es el máximo valor de los pedidos realizados durante el mismo día para cada uno de los clientes, teniendo en cuenta que sólo queremos mostrar aquellos pedidos que superen la cantidad de 2000 €.
7. Calcula el máximo valor de los pedidos realizados para cada uno de los comerciales durante el año 2016. Muestra el identificador del comercial, nombre, apellidos y total.
8. Devuelve un listado con el identificador de cliente, nombre y apellidos y el número total de pedidos que ha realizado cada uno de clientes. Tenga en cuenta que pueden existir clientes que no han realizado ningún pedido. Estos clientes también deben aparecer en el listado indicando que el número de pedidos realizados es 0.
9. Devuelve un listado con el identificador de cliente, nombre y apellidos y el número total de pedidos que ha realizado cada uno de clientes durante el año 2017.
10. Devuelve un listado que muestre el identificador de cliente, nombre, primer apellido y el valor de la máxima cantidad del pedido realizado por cada uno de los clientes. El resultado debe mostrar aquellos clientes que no han realizado ningún pedido indicando que la máxima cantidad de sus pedidos realizados es 0 . Puede hacer uso de la función IFNULL.

La función IFNULL contiene dos parámetros, devolverá el primero si no es NULL, y si es NULL devuelve el segundo.

Ejemplos:

IFNULL(total, 0). Si el contenido de total es NULL devuelve 0, si es distinto devuelve total.

SELECT IFNULL(NULL, 'nada') devuelve 'nada'.

SELECT IFNULL('hola', 'nada') devuelve 'hola'.

11. Devuelve cuál ha sido el pedido de máximo valor que se ha realizado cada año.
12. Devuelve el número total de pedidos que se han realizado cada año.